TARVI: Methodology Notes

From the splicing ODE to velocity, latent time, and the training objective

Technical companion document

Abstract

These notes give a self-contained account of how TARVI works: the biophysical model it rests on, the exact solutions of that model, what the data can and cannot identify, how the neural architecture is organised, how the model is trained without any ground-truth velocity, and how velocity and latent time are finally computed. Every equation is derived rather than quoted, and every component is tied back to the code that implements it. The document is intended to answer, in one place, the questions that arise naturally when reading the implementation — in particular: *where do the equations come from*, *how are unknown kinetic rates obtained*, *why is there an encoder and decoder at all*, and *what is the loss computed against when no ground truth exists*.

Contents

1 Introduction

1.1 What RNA velocity is

Messenger RNA passes through a short pipeline inside every cell. A gene is transcribed into unspliced pre-mRNA, introns are spliced out to give mature mRNA, and mature mRNA is eventually degraded:

$$\text{gene} \xrightarrow{\alpha} \underbrace{u}_{\text{unspliced}} \xrightarrow{\beta} \underbrace{s}_{\text{spliced}} \xrightarrow{\gamma} \varnothing \tag{1}$$

with three rate constants: α (transcription), β (splicing) and γ (degradation of spliced mRNA). Note that *translation does not appear*. Protein is not measured in scRNA-seq, so it plays no role in the model; γ is a degradation rate, not a translation rate.

Modern droplet-based protocols report unspliced and spliced counts separately. *RNA velocity* is the instantaneous rate of change of the mature pool:

$$v \;=\; \frac{ds}{dt} \;=\; \beta u - \gamma s \tag{2}$$

Mature mRNA rises when it arrives faster than it is destroyed.

A positive v means the gene is being switched on and the cell is moving toward a state in which that gene is more highly expressed. Computing v for every gene yields one vector per cell — an arrow pointing at the cell's near future. Projected onto an embedding, those arrows reconstruct a differentiation trajectory from a single static snapshot, with no time-course experiment required.

1.2 What TARVI adds

Contemporary velocity estimators share three structural limitations, and TARVI targets each one directly.

1. **Static transcription rate.** The per-gene rate α_g is treated as a constant, ignoring that genes are switched on by transcription factors whose abundance differs from cell to cell. TARVI makes α *cell-specific*, computed from a mask of known TF–target relationships derived from ENCODE ChIP-seq and ChEA.

2. **Weakly supervised latent time.** Latent time falls out of the fit with no direct signal, and is consequently poorly calibrated. TARVI adds a supervised residual time head trained by two complementary losses.

3. **Local/global mismatch.** Velocity is a local derivative; a trajectory is a global object. TARVI blends the learned time with diffusion pseudotime computed on the velocity transition graph.

1.3 How to read these notes

Section 2 derives the kinetic model and its exact solutions from first principles. Section 3 establishes what the data can and cannot determine — this underpins several later design choices and is worth reading before the architecture. Section 4 covers data preparation. Section 5 describes the network. Section 6 explains how training proceeds without ground truth. Section 7 covers velocity and latent-time computation at inference. Appendix A is a notation and tensor-shape reference; readers may find it useful to keep open alongside the text.

2 The kinetic model and its exact solutions

2.1 Mass-action kinetics

The three processes give a coupled pair of ordinary differential equations:

$$\frac{du}{dt} = \underbrace{\alpha}_{\text{transcribed}} - \underbrace{\beta u}_{\text{spliced away}} \tag{3}$$

$$\frac{ds}{dt} = \underbrace{\beta u}_{\text{arrives}} - \underbrace{\gamma s}_{\text{degraded}} \tag{4}$$

The form of each term follows from elementary reaction kinetics:

- α is a *constant* (zeroth order): the rate at which polymerase produces transcripts does not depend on how much RNA is already present.
- βu is *first order*: twice as much unspliced RNA means twice as many splicing events per unit time.
- γs is likewise first order for degradation.

The coupling matters. The term βu is subtracted in (3) and added in (4): nothing is lost in transit, it is the same molecules moving from one pool to the other. This conservation is what makes the unspliced pool a *leading indicator* of the spliced pool, and is the entire basis of RNA velocity.

2.2 Induction: solving for u

Equation (3) is a first-order linear ODE. Write it in standard form and multiply by the integrating factor $e^{\beta t}$:

$$\frac{du}{dt} + \beta u = \alpha \qquad \Longrightarrow \qquad e^{\beta t}\frac{du}{dt} + \beta e^{\beta t} u = \alpha e^{\beta t}. \tag{5}$$

The left-hand side is now exactly the product rule in reverse, so

$$\frac{d}{dt}\Big(u\, e^{\beta t}\Big) = \alpha e^{\beta t}. \tag{6}$$

Integrating both sides and dividing through by $e^{\beta t}$,

$$u\, e^{\beta t} = \frac{\alpha}{\beta} e^{\beta t} + C \qquad \Longrightarrow \qquad u(t) = \frac{\alpha}{\beta} + C e^{-\beta t}. \tag{7}$$

Imposing the induction initial condition $u(0) = 0$ (the gene starts from off) gives $C = -\alpha/\beta$, hence

$$\boxed{u(t) = \frac{\alpha}{\beta}\Big(1 - e^{-\beta t}\Big)} \tag{8}$$

Two sanity checks: $u(0) = 0$ as required, and $u(t) \to \alpha/\beta$ as $t \to \infty$, which is the steady-state level.

2.3 Induction: solving for s

Substituting (8) into (4) gives a forced linear ODE in s:

$$\frac{ds}{dt} + \gamma s = \beta \cdot \frac{\alpha}{\beta}\Big(1 - e^{-\beta t}\Big) = \alpha - \alpha e^{-\beta t}. \tag{9}$$

Applying the integrating factor $e^{\gamma t}$,

$$\frac{d}{dt}\Big(s\, e^{\gamma t}\Big) = \alpha e^{\gamma t} - \alpha e^{(\gamma-\beta)t}, \tag{10}$$

and integrating,

$$s\, e^{\gamma t} = \frac{\alpha}{\gamma} e^{\gamma t} - \frac{\alpha}{\gamma - \beta} e^{(\gamma-\beta)t} + C. \tag{11}$$

Dividing by $e^{\gamma t}$ and imposing $s(0) = 0$, which fixes $C = \frac{\alpha}{\gamma-\beta} - \frac{\alpha}{\gamma}$, and collecting terms:

$$\boxed{s(t) = \frac{\alpha}{\gamma}\Big(1 - e^{-\gamma t}\Big) + \frac{\alpha}{\gamma - \beta}\Big(e^{-\gamma t} - e^{-\beta t}\Big)} \tag{12}$$

Again $s(0) = 0 + \frac{\alpha}{\gamma-\beta}(1-1) = 0$, and $s(t) \to \alpha/\gamma$ as $t \to \infty$.

The second term in (12) is the **lag term**. It is what makes s trail behind u, and therefore what makes the (s, u) trajectory trace a loop rather than a straight line. Everything downstream — the identifiability of the rates, the gene filter, the initialisation — follows from the existence of that lag.

2.4 Repression

At the switch time the gene is turned off, so $\alpha = 0$. The equations are the same but with new initial conditions (u_0, s_0), namely the values reached at the end of induction. Equation (3) becomes pure decay:

$$\frac{du}{dt} = -\beta u \qquad \Longrightarrow \qquad \boxed{u(t) = u_0\, e^{-\beta t}} \tag{13}$$

For s, the forcing term is now $\beta u_0 e^{-\beta t}$:

$$\frac{d}{dt}\Big(s\, e^{\gamma t}\Big) = \beta u_0 e^{(\gamma-\beta)t} \qquad \Longrightarrow \qquad s(t) = \frac{\beta u_0}{\gamma - \beta} e^{-\beta t} + C e^{-\gamma t}. \tag{14}$$

Imposing $s(0) = s_0$ gives $C = s_0 - \frac{\beta u_0}{\gamma-\beta}$, hence

$$\boxed{s(t) = s_0 e^{-\gamma t} - \frac{\beta u_0}{\gamma - \beta}\Big(e^{-\gamma t} - e^{-\beta t}\Big)} \tag{15}$$

Equations (8), (12), (13) and (15) are implemented term for term by the induction and repression solvers in `tarvi/_module.py`. Because they are closed-form, no numerical integration is ever performed: evaluating the model at any time is a handful of exponentials, fully differentiable and cheap.

2.5 Steady states and the phase portrait

Setting both derivatives to zero,

$$\alpha - \beta u^* = 0 \;\Rightarrow\; u^* = \frac{\alpha}{\beta}, \qquad \beta u^* - \gamma s^* = 0 \;\Rightarrow\; s^* = \frac{\alpha}{\gamma}. \tag{16}$$

These are the two *steady* states of the four-state model: the induction plateau $(\alpha/\beta,\ \alpha/\gamma)$ and, trivially, the repression floor $(0, 0)$.

Dividing one by the other gives a relation that will recur throughout these notes:

$$\frac{u^*}{s^*} = \frac{\gamma}{\beta} \tag{17}$$

The steady-state slope of the phase portrait equals γ/β.

The **phase portrait** is the scatter of (s, u) across all cells for a single gene. During induction u rises first and s lags, producing an upper-left arm; during repression u falls first and s lags again, producing a lower-right arm. Together they trace a counter-clockwise loop whose diagonal has slope γ/β by (17). This geometry is the sole source of information about the kinetic rates, as Section 3 makes precise.

Figure 1 shows both views of the same gene: the time courses on the left, and the phase portrait they trace on the right.

2.6 A removable singularity at $\gamma = \beta$

Both expressions for s divide by $(\gamma - \beta)$. When a gene has $\gamma \approx \beta$ the expression is numerically unstable, and at exact equality it is undefined. Taking the limit $\gamma \to \beta$ in (12) (write $\gamma = \beta + \varepsilon$ and expand to first order in ε) gives the repeated-root solution

$$s(t) \;\xrightarrow[\gamma\to\beta]{}\; \frac{\alpha}{\beta}\Big(1 - e^{-\beta t}\Big) - \alpha\, t\, e^{-\beta t}, \tag{18}$$

which is finite. The singularity is therefore removable. The implementation does not branch on this case; it adds a small constant ($\texttt{eps} = 10^{-6}$) to the denominator, which is adequate in practice since exact equality has measure zero.

3 What the data can and cannot tell us

A question that arises immediately on reading the model is: *the splicing and degradation rates were never measured, so how can we use them?* This section answers that, and establishes two limitations that shape the rest of the design.

3.1 The rates are inferred, not measured

β and γ are not experimental quantities supplied to the model. They are **free parameters**, one per gene, declared as `torch.nn.Parameter` and learned by gradient descent:

$$\beta_g = \text{clamp}\big(\text{softplus}(\theta_g^{\beta}),\, 0,\, 50\big), \qquad \gamma_g = \text{clamp}\big(\text{softplus}(\theta_g^{\gamma}),\, 0,\, 50\big). \tag{19}$$

The softplus enforces positivity — negative rates are physically meaningless — and the clamp keeps values in a range where the exponentials in (8)–(15) do not overflow.

They are learned because the ODE solutions are the *means of the likelihood*. When observed u and s are scored against that likelihood and the error is backpropagated, the gradient moves β and γ toward values that make the predicted curve pass through the observed data cloud. This is ordinary maximum likelihood: an unmeasured quantity is recovered from the pattern it leaves behind in quantities that *were* measured.

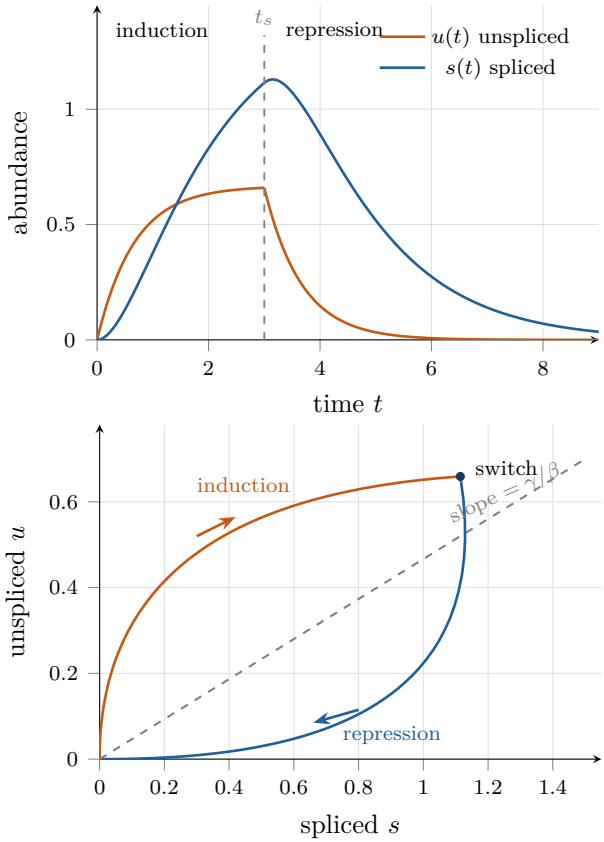

Figure 1: Two views of one gene, computed from the exact solutions (8)–(15) with $\alpha = 1$, $\beta = 1.5$, $\gamma = 0.7$, $t_s = 3$. **Left:** time courses. Unspliced rises first and falls first; spliced trails it, and continues rising briefly *after* the gene has switched off. That lag is the second term of (12). **Right:** the same trajectory drawn as (s, u), which is what a scatter over cells reveals. The lag turns the path into a counter-clockwise loop; the induction arm sits above the steady-state diagonal and the repression arm below it. The dashed diagonal has slope γ/β by (17) and is the only feature of the data that identifies the rates.

3.2 What identifies them: the phase portrait

Two features of the loop described in Section 2.5 carry information about the rates:

Loop feature	**What it determines**
Slope of the steady-state diagonal	the ratio γ/β, via (17)
Width / curvature of the loop	how *different* β and γ are (the lag)

A gene whose phase portrait is a shapeless blob carries *no* information about its own kinetics. This is the entire justification for the gene filter described in Section 4.2.

3.3 Scale invariance

There is a fundamental limit on what can be recovered.

Proposition 1 (Scale non-identifiability)**.** For any $c > 0$, the reparameterisation

$$\alpha \to c\alpha, \qquad \beta \to c\beta, \qquad \gamma \to c\gamma, \qquad t \to t/c$$

leaves $u(t)$ and $s(t)$ exactly unchanged.

Proof. Substituting into (8),

$$\frac{c\alpha}{c\beta}\Big(1 - e^{-c\beta \cdot t/c}\Big) = \frac{\alpha}{\beta}\Big(1 - e^{-\beta t}\Big),$$

and identically for (12), since every rate appears either as a ratio of two rates or multiplied by t. The same holds for the repression branch. □

The data therefore cannot distinguish between a slow process observed over a long interval and a fast process observed over a short one. Only *ratios* of rates are identifiable. Two consequences follow, and both are visible in how results are reported.

Consequence 1: the time scale is a convention. The model fixes the remaining degree of freedom by setting $t_{\max} = 20$ and initialising $\beta \approx 1$, so all rates are effectively expressed in units of the splicing rate. Latent time is therefore in arbitrary units, and only its *ordering* is meaningful. This is why evaluation reports Spearman correlation of pseudotime rather than absolute error.

Consequence 2: velocity magnitude is arbitrary, direction is not. Under the same rescaling $v \to c\,v$ by (2). The length of every arrow is defined only up to one global constant, but its direction is invariant. This is why every reported metric — cross-boundary direction accuracy, velocity confidence, cosine-based coherence — is direction-based.

3.4 What keeps the fit well behaved

Because the problem is unidentifiable in general and non-convex in practice, several components exist specifically to constrain it:

- the **end penalty**, which forces the induction curve evaluated at the switch time to land on a data-derived anchor, pinning down where induction ends and thereby constraining γ/β;
- the **sparse Dirichlet prior** on the state weights, which forces confident state assignment and blocks degenerate fits that hedge across all four states;

- the **ODE residual loss**, which ties latent time to observed expression so that the rates cannot drift to compensate for a bad time estimate;
- **data-driven initialisation** (Section 4.3), which places the optimiser in a sensible basin from the start.

4 Data preparation

4.1 Moments and scaling

Three preprocessing steps precede training.

1. **Moments.** First-order moments M_s, M_u are computed upstream: each cell's counts are averaged over its k-nearest-neighbour neighbourhood. Raw droplet counts are far too sparse — dominated by dropout zeros — to fit a smooth ODE against.
2. **Min–max scaling** of M_s and M_u to $[0, 1]$. This is required because the likelihood uses a Gaussian with a shared per-gene scale; without rescaling, highly expressed genes would dominate the reconstruction term entirely.
3. **Gene filtering**, described next.

4.2 The gene filter

A throwaway steady-state fit is run purely to compute a screening statistic:

```
scv.tl.velocity(adata, mode="deterministic")
adata = adata[:, np.logical_and(adata.var.velocity_r2 > 0,
                                adata.var.velocity_gamma > 0)].copy()
adata = adata[:, adata.var.velocity_genes].copy()
```

The `deterministic` mode is the original steady-state estimator: for each gene independently it fits a straight line $u \approx \gamma s$ through the extreme-quantile cells (those assumed to be at steady state, cf. equation (17)). Two quantities result per gene: `velocity_gamma`, the fitted slope, and `velocity_r2`, the coefficient of determination

$$R^2 = 1 - \frac{\mathrm{SS}_{\mathrm{residual}}}{\mathrm{SS}_{\mathrm{total}}}. \tag{20}$$

The interpretation of the threshold is as follows:

Value	Meaning
$R^2 = 1$	the line passes through every point exactly
$R^2 = 0$	the line is no better than predicting the mean of u
$R^2 < 0$	the line is *worse* than predicting the mean

Negative R^2 is possible — and is precisely what the filter targets — because the line is fitted on a *subset* of cells (the extreme quantiles) and then scored on *all* cells. A gene with no coherent phase portrait yields an arbitrarily placed line that scores worse than a flat mean. The condition $R^2 > 0$ is thus a deliberately minimal bar: the steady-state line must explain *something*.

The companion condition $\gamma > 0$ rejects a negative fitted slope, which would imply that unspliced and spliced counts anti-correlate — impossible under (4), as it would require a negative degradation rate. The third filter applies scVelo's own composite criterion, which additionally requires sufficient detection across cells.

Retaining genes that fail these tests would cost twice over: model capacity is spent fitting an ODE to noise, which given the non-convexity can drag the optimiser into a poor basin; and because velocity direction is computed across genes, uninformative genes dilute the signal in every cell's arrow.

A note on circularity. This is a pre-filter that uses a simpler model to select genes for a more complex one. The bar is deliberately permissive (> 0, not > 0.5) so that it removes genes carrying no signal rather than selecting genes that happen to suit the steady-state model's assumptions.

4.3 Initialisation

Because these ODEs admit many poor local optima, initialisation is consequential. For each gene, the median over the top 1% of cells by unspliced count is taken, giving $(u_{\text{upper}}, s_{\text{upper}})$: these cells sit near the top of induction, close to the steady state. They supply three initial values:

Quantity	**Initialisation**
α	$\text{softplus}^{-1}(u_{\text{upper}})$ since $u^* = \alpha/\beta$ with $\beta \approx 1$
switch anchor (u_0, s_0)	$(u_{\text{upper}}, s_{\text{upper}})$
γ	$u_{\text{upper}}/s_{\text{upper}}$ the steady-state slope, eq. (17)
β	$\text{softplus}(0.5) \approx 0.97$, i.e. effectively 1

Setting $\beta \approx 1$ is the concrete act of fixing the scale degree of freedom identified in Proposition 1.

5 Model architecture

5.1 Why a variational autoencoder?

The natural question is why a neural network is involved at all, when the underlying model is a small ODE with a handful of parameters. The answer is a parameter-counting argument.

Fitting the model requires, for every cell and every gene, a position in the kinetic cycle: the induction progress ρ, the repression progress τ, and four state probabilities π. For $N = 3000$ cells and $G = 1000$ genes that is on the order of 1.8×10^7 free parameters fitted against 6×10^6 observations. The problem is hopelessly overparameterised, and — just as importantly — every cell would be fitted in isolation, with no sharing of information between similar cells.

Amortised inference replaces those per-cell parameters with a *function*. An encoder maps a cell's observed profile to a low-dimensional state z; a decoder maps z to the per-gene kinetic variables. The model then carries roughly 10^5 shared network weights instead of 10^7 per-cell parameters, and every cell's estimate is informed by every other cell because all pass through the same weights.

Note the final arrow. **The decoder does not predict expression.** It predicts *ODE parameters*, which then pass through the fixed analytic solutions of Section 2. The physics is imposed, not learned. This is what keeps the model mechanistic and its outputs kinetically interpretable, rather than a black-box autoencoder that happens to reconstruct well.

Two further benefits follow from the choice of a *variational* autoencoder over a plain one. The KL term regularises the latent space to be smooth and continuous, so nearby cells receive nearby codes; and because z is sampled rather than deterministic, repeated forward passes give a posterior distribution over every downstream quantity, yielding uncertainty estimates rather than point estimates.

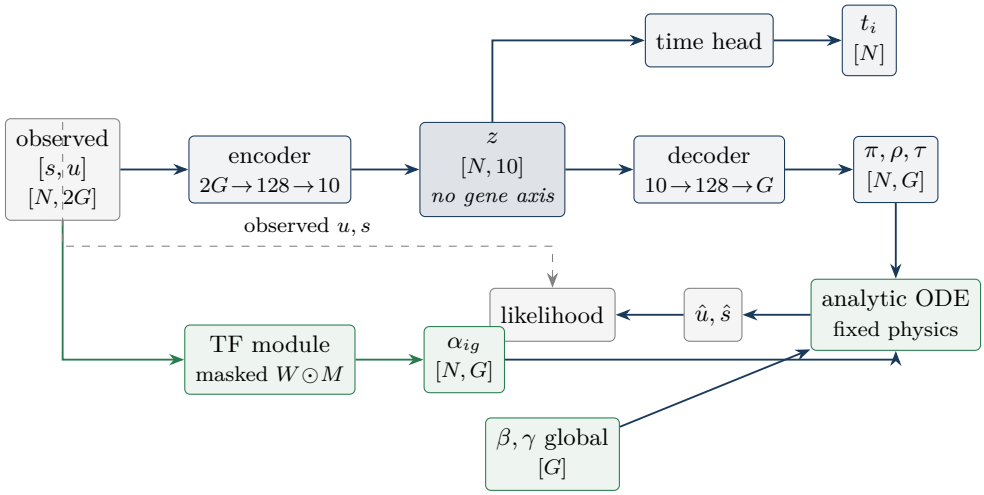

Figure 2: Data flow. The encoder compresses the full expression profile into a gene-free cell state z; the decoder re-creates the gene axis as kinetic variables; the analytic ODE — *fixed physics, not learned* — turns those into predicted abundances that are scored against the observation. Two quantities bypass the bottleneck: the global rates β, γ, and (in green) the transcription rate α, which is read directly from the cell's own transcription-factor expression rather than from z.

5.2 The encoder, and why z is cell-level

The encoder receives the concatenation of spliced and unspliced profiles and returns a variational posterior:

$$q(z \mid s, u) = \mathcal{N}\big(\mu_z(s,u),\, \sigma_z^2(s,u)\big), \qquad z \in \mathbb{R}^{10}. \tag{21}$$

The claim that z is a *cell-level* quantity is not an interpretation but a statement about tensor axes. Table 1 classifies every quantity in the model.

Table 1: Every quantity in TARVI, classified by the axes it carries. N = cells, G = genes.

Quantity	**Shape**	**Axes**	**Level**
β, γ, t_s	$[G]$	gene only	gene-level
z	$[N, 10]$	cell only	**cell-level**
`learned_time`	$[N]$	cell only	**cell-level**
ρ, τ	$[N, G]$	cell × gene	cell–gene
α (TF-regulated)	$[N, G]$	cell × gene	cell–gene
π	$[N, G, 4]$	cell × gene × state	cell–gene

z has no gene axis. Tracing the computation makes clear why. The encoder's first layer is dense, mapping $2G \to 128$: every gene connects to every hidden unit and no per-gene pathway survives. The gene axis is destroyed on the first matrix multiplication. Symmetrically, the decoder's layers map $10 \to 128 \to G$: *the gene axis is created there*, and does not exist upstream.

It is worth addressing the natural objection — that z is computed *from* gene expression and so ought to be gene-level. Being computed from an axis is not the same as being indexed by it. A student's grade-point average is computed from every course grade, but is a student-level quantity; "the GPA of Chemistry" is a malformed question because the course axis was summed away. Likewise "the z of gene *Sox9*" names no object. The ten coordinates of z are learned abstract factors — position along a trajectory, lineage branch, cell-cycle phase — not

gene slots.

Two structural pressures force z to be a genuinely global summary. First, the bottleneck: ten numbers cannot store per-gene information about a thousand genes, so only what is shared across genes survives compression. Second, the same z must explain *all* genes simultaneously through the decoder; anything gene-specific would be useless to the other $G - 1$ genes and is squeezed out during training. This second pressure is exactly what couples the genes together and yields a coherent cell-level trajectory rather than G independent gene clocks.

5.3 The decoder and the four kinetic states

The decoder maps z to three per-cell, per-gene quantities:

Output	Shape	Meaning
π_α	$[N, G, 4]$	Dirichlet concentrations over kinetic states
ρ	$[N, G] \in (0, 1)$	fractional progress through induction
τ	$[N, G] \in (0, 1)$	fractional progress through repression

5.3.1 Two trunks and three heads

Structurally the decoder is a pair of independent multilayer trunks, each reading z, followed by three output heads:

```
# two independent trunks, both reading z
self.rho_first_decoder = FCLayers(n_in=10, n_out=128)    # serves rho
    and tau
self.pi_first_decoder  = FCLayers(n_in=10, n_out=128)    # serves pi

# three output heads
self.px_rho_decoder = nn.Sequential(nn.Linear(128, G), nn.Sigmoid())
self.px_tau_decoder = nn.Sequential(nn.Linear(128, G), nn.Sigmoid())
self.px_pi_decoder  = nn.Linear(128, 4 * G)
```

Here `FCLayers` is the scvi-tools block — Linear, batch norm, layer norm, ReLU and dropout. The progress variables ρ and τ share a trunk, since both describe a position within a phase and can reuse the same features, but receive separate heads because they describe *different* phases. The state weights are given their own trunk entirely.

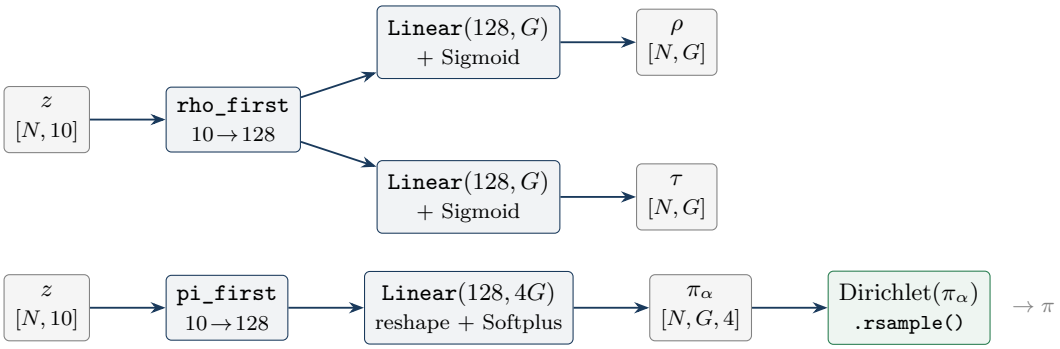

Figure 3: Decoder structure. Two trunks map the ten-dimensional cell state to 128-dimensional hidden vectors — still with no gene axis. The output heads create the gene axis, each of the G output units being a distinct learned readout of the same shared hidden vector.

5.3.2 Where the gene axis is created

This is the mechanical counterpart to Section 5.2. The trunks emit a 128-dimensional vector *per cell*; there are still no genes. The gene axis is manufactured by the output layers,

$$\underbrace{h_\rho \in \mathbb{R}^{128}}_{\text{per cell}} \xrightarrow{W \in \mathbb{R}^{G \times 128}} \underbrace{\rho \in \mathbb{R}^{G}}_{\text{per cell, per gene}} , \tag{22}$$

in which each of the G output units is a different learned linear readout of the same shared hidden vector. Gene g's row of W poses its own question of the cell state. This is how a single ten-dimensional code can specify positions for a thousand genes: the genes differ in *how they read* the cell state, not in what state information reaches them.

5.3.3 Why sigmoid on ρ and τ

Neither variable is a time. Both are *fractions of a phase*, converted to times by scaling:

$$t_{\text{ind}} = t_s \cdot \rho \in (0, t_s), \qquad t_{\text{rep}} = (t_{\max} - t_s) \cdot \tau \in (0, t_{\max} - t_s). \tag{23}$$

This is a parameterisation device rather than a cosmetic choice. The alternative — predicting an absolute time and hoping it falls inside the correct phase — would require a penalty term and could still be violated. Here the constraint is *structural*: the sigmoid bounds the output to $(0, 1)$, and multiplying by the phase width makes it arithmetically impossible for an induction time to exceed the switch point. No loss term is needed and no invalid state is reachable.

A second consequence is that ρ and τ are *relative* coordinates. A cell with $\rho = 0.5$ is halfway through induction *for that gene*, whether that gene's switch time is 2 or 18.

5.3.4 Why softplus on π: concentrations are not probabilities

The decoder does not emit probabilities. It emits Dirichlet *concentrations*, which are positive and unnormalised; the probabilities appear one step later, by sampling:

```
px_pi_alpha = Softplus(reshape(px_pi_decoder(h_pi), (N, G, 4)))  #
    decoder output
px_pi       = Dirichlet(px_pi_alpha).rsample()                   #
    what is used
```

Softplus enforces positivity, which is the Dirichlet's only requirement on its parameters. The distinction between the two objects matters:

	Shape	**Constraint**	**Meaning**
π_α (decoder output)	$[N, G, 4]$	> 0	concentrations: both the preferred state *and* the confidence
π (after `rsample`)	$[N, G, 4]$	sums to 1	the state weights entering the mixture

Two things follow. First, **confidence is represented separately from preference**: the concentrations $(10, 1, 1, 1)$ and $(1.0, 0.1, 0.1, 0.1)$ induce the same mean probabilities but very different certainty, a distinction a plain softmax cannot express since it retains only the ratio. Second, `rsample` is the *reparameterised* Dirichlet draw, so gradients pass through the state assignment. Sampling is ordinarily a barrier to backpropagation; here it is not, which is why no `argmax` appears anywhere in the model — the state assignment remains a differentiable mixture throughout.

5.3.5 What the decoder deliberately does not produce

Quantity	Source	Why not the decoder
β, γ	global parameters $[G]$	properties of the gene, not of the cell; no cell axis is required
α	TF module, read from data	requires specific TF identity, which cannot survive the ten-dimensional bottleneck
t_s	global parameter $[G]$	a property of the gene's kinetics, shared by all cells

The division of labour is therefore clean: **the decoder answers *where* a cell sits in each gene's cycle; the rate parameters answer *how fast* that cycle runs.**

A subtlety in the implementation. The two trunks are not fed identically:

```
rho_first = self.rho_first_decoder(z_in)    # z_in - may be masked
pi_first  = self.pi_first_decoder(z)        # z    - always full
```

Here `z_in` is the latent-dimension-masked variant used by the interpretability path, which restricts the decoder to a single latent dimension in order to ask what that dimension encodes. The masking therefore applies to ρ and τ but not to π. This is harmless during ordinary training, where the two are identical, but is worth knowing before running per-dimension attribution: the state assignment will not respond to the mask.

5.3.6 The four states

Writing t_s for the learned per-gene switch time, the four states and their times are:

State	Time	Predicted (u, s)
Induction	$t = t_s\,\rho$	eqs. (8), (12)
Induction steady	—	$(\alpha/\beta,\ \alpha/\gamma)$
Repression	$t = (t_{\max} - t_s)\,\tau$	eqs. (13), (15)
Repression steady	—	$(0, 0)$

Each state contributes a Gaussian centred on its predicted (u, s) with a learned per-gene scale; these are combined by `MixtureSameFamily` weighted by the sampled π. The repression-steady component receives a scale reduced by a factor of ten, since it is an exact point rather than a fitted curve.

5.4 Contribution 1: TF-regulated transcription

5.4.1 Building the prior

A binary mask of known regulatory relationships is assembled from ENCODE ChIP-seq and ChEA 2016. TF–target pairs are parsed from both sources, gene names are resolved case-insensitively against the dataset so that only TFs and targets actually present survive, evidence is counted per pair, and the top 99 TFs per target gene are retained. The result is a mask $M \in \{0, 1\}^{G \times T}$ together with initial weights given by the Spearman correlation between each TF's and target's expression, computed over cells where both exceed a small threshold and scaled by 0.1.

5.4.2 The cell-specific rate

The transcription rate is then computed per cell:

$$\alpha_{ig} = \mathrm{clamp}\Big(\mathrm{softplus}\Big(\textstyle\sum_k \big(w_{gk} \odot M_{gk}\big)\,\mathrm{TF}_k(i) + b_g\Big),\ 0,\ 50\Big) \tag{24}$$

where b_g is a learnable basal rate warm-started from the static per-gene α, and $\mathrm{TF}_k(i)$ is the spliced expression of the k-th transcription factor in cell i, read directly from the data rather than from z.

Three points are worth emphasising. The mask is applied *inside* the forward pass, so gradients for non-edges are structurally zero and the model cannot invent regulation — it learns only the strength and sign of edges that ChIP-seq evidence already supports. The static α is frozen when TF regulation is active, so there is no competing pathway. And a mild L_1 penalty on W ensures each gene ends up driven by a few dominant regulators rather than all 99, which makes the fitted W readable as a regulatory network.

Comparing with the baseline: where VeloVI has α_g, a single number per gene shared by all cells, TARVI has α_{ig}, an $[N \times G]$ matrix that varies across the manifold according to which transcription factors each cell expresses.

5.4.3 How α enters the ODE

The transcription rate appears in exactly three places, and in every one of them it appears as a *multiplicative factor*:

Location	**Role of α**
Induction curve, eqs. (8), (12)	overall amplitude
Induction steady state	the plateau ($\alpha/\beta,\ \alpha/\gamma$)
Repression, eqs. (13), (15)	only via (u_0, s_0), since $\alpha = 0$ there

This is clearest when the induction solution is factored:

$$u(t) = \alpha \cdot \underbrace{\tfrac{1}{\beta}\big(1 - e^{-\beta t}\big)}_{\text{shape — independent of } \alpha}. \tag{25}$$

α sets the height of the curve; β and γ set its shape; t sets the position along it. These are three separate roles, and the distinction is what the next subsection turns on.

5.4.4 Velocity is directly proportional to α

Substituting the induction solutions into $v = \beta u - \gamma s$, the terms collapse. With $\beta u = \alpha(1 - e^{-\beta t})$ and $\gamma s = \alpha(1 - e^{-\gamma t}) + \frac{\alpha\gamma}{\gamma-\beta}(e^{-\gamma t} - e^{-\beta t})$,

$$v_{\mathrm{ind}} = \alpha\big(e^{-\gamma t} - e^{-\beta t}\big)\Big[1 - \tfrac{\gamma}{\gamma-\beta}\Big] = \alpha\big(e^{-\gamma t} - e^{-\beta t}\big)\Big[\tfrac{-\beta}{\gamma-\beta}\Big], \tag{26}$$

that is,

$$\boxed{v_{\mathrm{ind}}(t) = \frac{\alpha\beta}{\gamma - \beta}\Big(e^{-\beta t} - e^{-\gamma t}\Big)} \tag{27}$$

Induction velocity is exactly proportional to the transcription rate.

The expression vanishes at $t = 0$ and again as $t \to \infty$, peaking in between — correct for a gene that ramps up and then plateaus. The point of (27) is that α is not a peripheral nuisance parameter: it is the *amplitude of the velocity itself*. Doubling a cell's transcription rate exactly doubles its velocity for that gene at a fixed time.

5.4.5 Why a cell-specific α helps

It separates amplitude from time. With one α per gene, every cell lies on a *single shared curve*. The only mechanism available to explain a cell with high expression is that it has progressed further through induction. Genuine differences in regulatory drive are therefore silently re-encoded as differences in developmental time.

Figure 4 makes the consequence concrete. Two cells are observed at the identical value $u = 0.5$, but one has strong transcription-factor support and the other weak. Under (3) their instantaneous dynamics differ by a factor of fifteen:

	TF activity	α	inferred t	$du/dt = \alpha - \beta u$
Cell A	high	2.0	0.29	**1.5**
Cell B	low	0.6	1.79	**0.1**

A static-α model must assign these two cells the same time *and* the same velocity. It has no vocabulary for "these cells differ in regulatory drive rather than in developmental stage".

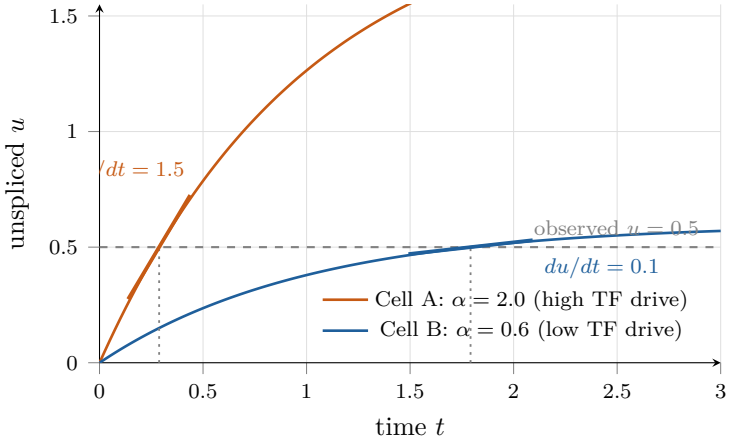

Figure 4: Why a cell-specific α matters. Both cells are measured at $u = 0.5$, but α scales the whole induction curve, so the same observation places them at very different points in the cycle: cell A is early on a tall curve and still climbing steeply, cell B is near the ceiling of a short curve and has almost stopped. The heavy segments are the tangents, whose slopes are the unspliced velocities — a factor of fifteen apart. A model with one α per gene has a single curve and must give both cells the same answer.

It costs almost no parameters. A freely fitted α_{ig} would introduce $N \times G$ parameters — entirely unidentifiable, capable of fitting any data exactly and therefore explaining nothing. Instead α_{ig} varies over all $N \times G$ entries while being *generated* by only $|M| + G$ parameters (the non-zero mask weights plus the basal rates), shared across every cell. The cell-to-cell variation is supplied by *measured transcription-factor expression*, not by free parameters: cell-specificity is bought with data rather than with degrees of freedom.

It is a second channel that bypasses the bottleneck. As established in Section 5.2, z has ten dimensions and therefore necessarily destroys gene-specific detail. The TF pathway reads transcription factor expression *directly from the data*, not through z — the green route in Figure 2. Specific regulatory identity, of the form "this particular cell strongly expresses this particular factor", therefore reaches α even though it could never survive compression into ten dimensions.

It yields a readable network. After training, $W \odot M$ is a gene-by-TF matrix of learned regulatory strengths with signs, sparsified by the L_1 penalty. This is a scientific output — an inferred regulatory network — obtained as a by-product of velocity estimation.

A note on attribution. The mechanism above is the design rationale for a cell-specific α. The component ablation, however, attributes most of the *pseudotime* improvement to the ODE residual loss and to velocity–pseudotime blending, with TF regulation contributing principally on the interpretability side. Both statements should be made; claiming the headline metric for this component would overstate what the ablation supports.

5.5 The latent time head

A small residual network maps the cell state to a scalar time:

$$h = \mathrm{ReLU}(W_1 z) + W_{\mathrm{skip}}\, z, \tag{28}$$

$$h = \mathrm{ReLU}\big(W_2\, \mathrm{LayerNorm}(h)\big), \tag{29}$$

$$t_i = \sigma(W_{\mathrm{out}} h) \in [0, 1]. \tag{30}$$

5.5.1 Why the input is z

Latent time must be one number per cell, and by Table 1 the only cell-indexed, gene-free representation available is z. Beyond that necessity, reading from z has four advantages: the head sees the *same* cell state that generates the kinetics, so time and the ODE parameters cannot disagree about what cell this is; z is already denoised by the encoder, so pseudotime does not inherit dropout noise; the KL term makes z continuous, so nearby cells automatically receive nearby times; and a $10 \to 1$ regression is learnable from a few thousand cells, where $2G \to 1$ would not be.

There is also a less obvious effect. Because the ODE residual loss (Section 6.4.2) reaches the head, and the head's input is z, the gradient continues into the encoder:

$$\mathcal{L}_{\mathrm{res}} \;\to\; t_i \;\to\; \text{time head} \;\to\; z \;\to\; \text{encoder}. \tag{31}$$

The supervision therefore *reshapes the latent space itself* to be temporally organised. This is a substantial part of why pseudotime improves, and it is the reason the companion loss detaches its *target* but not its prediction.

5.5.2 The skip connection

Rather than a single chain of layers, the cell state travels two routes which are then summed:

The lower branch bypasses the nonlinearity entirely. That is the defining idea of a skip (or *residual*) connection: information is given a shortcut around one or more layers instead of being forced through them.

Two variants exist. The classical form introduced with residual networks is $h = f(x) + x$, a literal identity shortcut, available only when input and output dimensions agree. Here they

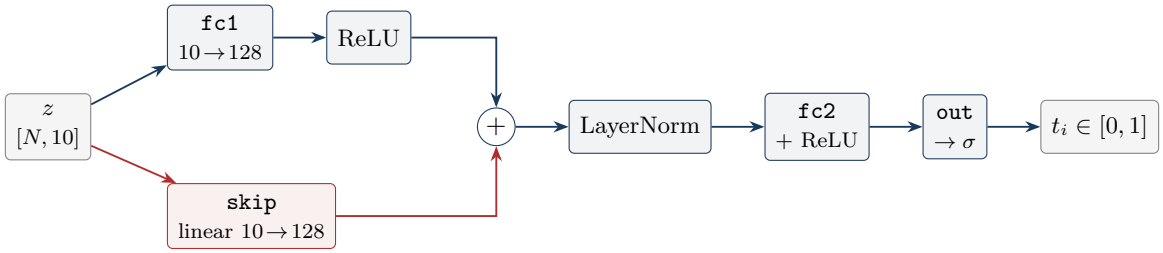

Figure 5: The time head. The upper branch passes through the nonlinearity; the lower branch (red) bypasses it with a plain linear map. Because their dimensions differ, the shortcut cannot be the identity and must itself be a learned projection.

do not — $\mathbb{R}^{10}$ against $\mathbb{R}^{128}$ — so a learned linear map stands in its place, $h = f(x) + W_{\text{skip}}x$. This is termed a *projection shortcut*.

The effect on the computation is small to write down but consequential:

Without skip	$h = \text{ReLU}(W_1 z)$
With skip	$h = \text{ReLU}(W_1 z) + W_{\text{skip}}\, z$

The term $\text{ReLU}(W_1 z)$ is non-negative and *exactly zero* on half of its input space. Adding $W_{\text{skip}}z$ contributes a plain linear component that is always active. The head therefore computes a linear function of z plus a nonlinear correction, rather than having to construct everything out of rectified pieces.

The question is not one of representational capacity. A plain multilayer perceptron is a universal approximator: $10 \to 128 \to 128 \to 1$ with rectified units can represent any smooth function of z, a linear one included. The skip therefore buys no expressiveness. What it buys is *what is easy to learn from a random initialisation*, and *what happens when training goes wrong.*

It encodes the right prior. Pseudotime is approximately linear in z: in a well-trained VAE the trajectory tends to lie along a principal direction of the latent space, so the ideal map is close to a linear ramp with corrections where the trajectory bends. Suppose that ideal map is $t \approx w^\top z$. The two architectures then face quite different learning problems:

Without skip	the weights must construct $w^\top z$ from scratch out of rectified pieces
With skip	W_{skip} *is* the linear part; W_1 learns only the deviation

Both reach the same final capacity, but by very different optimisation paths: the second starts near a sensible answer and refines it, whereas the first must first discover the linear structure before it can refine anything.

The failure mode here is ties, not error. Without the skip a ReLU network must *approximate* a straight line using piecewise-linear segments, which with a finite number of active pieces produces a staircase: flat plateaus joined by steps. This deserves emphasis because of how it interacts with the evaluation metric.

A staircase approximation to a ramp has *small squared error* — it is never far from the correct value — yet every flat segment assigns an identical time to many cells, and so destroys the ordering information within it. Since pseudotime is scored by Spearman correlation, a rank

statistic, the consequence is that **the training loss can look perfectly healthy while the reported metric collapses.** The linear bypass removes the staircase by construction, because a genuinely linear component always varies continuously with z.

It guarantees a gradient path. Differentiating the two branches,

$$\frac{\partial h}{\partial z} = \underbrace{\mathrm{ReLU}'(W_1 z)\, W_1}_{\text{zero wherever units are dead}} + \underbrace{W_{\text{skip}}}_{\text{always nonzero}} . \tag{32}$$

A rectified unit with negative pre-activation passes exactly zero gradient, and if enough units die the gradient reaching z collapses. This matters more here than in a typical network: as established in Section 5.5, the time head is the only conduit through which the ODE residual loss reaches z and reorganises the latent space. Were the head to close that path, nothing would raise an error — the pseudotime would simply, and silently, get worse.

It keeps the output sigmoid responsive. The final activation is $\sigma(\cdot)$, whose derivative vanishes at both extremes. If pre-activations drift large the gradient dies and every cell collapses to $t \approx 0$ or $t \approx 1$ — a genuine risk when several losses push simultaneously on one small head. This is also why the LayerNorm is placed *after* the addition rather than before it: summing two branches yields an output of unpredictable scale, and either branch could dominate the other by magnitude alone. Normalising the combined signal re-standardises it and holds the sigmoid in its responsive region.

Is it strictly necessary? No. A plain two-layer perceptron would in all likelihood train to something reasonable, and it would be an overstatement to claim the model fails without the shortcut.

The trade, however, is markedly lopsided. The skip costs a single 10×128 matrix — 1,280 parameters, negligible beside the rest of the model — and in exchange it protects a component whose failure is *silent* (Section 5.5: the head is the sole route by which the ODE residual loss reaches z) and whose downstream influence is large. That asymmetry is what justifies it: not evidence that the shortcut is required, but the observation that it is nearly free and closes a failure mode the training loss would not have revealed.

6 Training without ground truth

6.1 The principle

No ground-truth velocity, latent time, or kinetic rate is available for any dataset. This appears to make a loss function impossible: a loss compares an answer to a correct answer, and there is no correct answer to compare against.

The resolution is that **velocity is never optimised**. What is optimised is the reconstruction of the observed data:

```
reconst_loss_s = -mixture_dist_s.log_prob(spliced)      # target =
    observed s
reconst_loss_u = -mixture_dist_u.log_prob(unspliced)    # target =
    observed u
```

The model proposes rates and times; the ODE converts these into predicted $\hat{u}, \hat{s}$; those predictions are scored against what was actually measured. The observed counts *are* the ground truth. "Unsupervised" here means *no external labels*, not *no target*.

Velocity then follows for free, because by (2) it is a deterministic function of quantities the reconstruction has already pinned down. The same applies to latent time, state assignment, and the TF weights: all are *latent variables inferred as a by-product* of explaining the data. A quantity that is never directly optimised needs no label.

The bathtub picture. Two connected tanks: a tap fills the first at rate α, a pipe drains it into the second at rate β, and a drain empties the second at rate γ. The taps are hidden; only the water levels are visible. The model guesses tap settings, predicts the resulting levels, and is corrected by the discrepancy against the measured levels. Velocity — is the second tank filling or draining — is then simple arithmetic on the recovered taps.

6.2 A worked example

Consider one cell and one gene. Suppose the measured values are $u = 0.80$ and $s = 0.30$.

Round 1. The model's current guess is $\alpha = 1$, $\beta = 1$, $\gamma = 2$, with this cell at $t = 0.5$ in the induction state. Substituting into (8) and (12):

$$\hat{u} = \tfrac{1}{1}\big(1 - e^{-0.5}\big) = 0.39, \tag{33}$$

$$\hat{s} = \tfrac{1}{2}\big(1 - e^{-1}\big) + \tfrac{1}{2-1}\big(e^{-1} - e^{-0.5}\big) = 0.08. \tag{34}$$

	predicted	measured	error
u	0.39	0.80	-0.41
s	0.08	0.30	-0.22
		squared error	**0.21**

Both predictions are too low, so the gradient increases α.

Round 2. With $\alpha = 2$ and the other parameters unchanged:

$$\hat{u} = 0.79, \qquad \hat{s} = 0.15. \tag{35}$$

	predicted	measured	error
u	0.79	0.80	-0.01
s	0.15	0.30	-0.15
		squared error	**0.021**

The loss has fallen by an order of magnitude. Repeating this across every cell, every gene, for 500 epochs, is the whole training loop.

And then velocity. It was never part of the loss. It is computed afterwards from the recovered rates:

$$v = \beta u - \gamma s = 1 \times 0.79 - 2 \times 0.15 = +0.48, \tag{36}$$

positive, so this gene is being switched on in this cell. No ground-truth velocity was required at any point, because velocity is arithmetic on parameters that were themselves checked against measured expression.

Table 2: The training objective. Only the first three terms touch data; the remainder encode assumptions or priors.

Term	Supervised by	Weight	Kind
Reconstruction NLL	observed u, s	1.0	data
ODE residual	observed u, s (via t_i)	1.0	data
End penalty	top-1% quantile of the data	annealed	data-derived anchor
Time reconstruction	the model's own ODE time (stop-grad)	1.0	self-distillation
Velocity–time consistency	arrows point forward in time	0.2	structural prior
Temporal smoothness	similar cells share similar times	0.1	structural prior
Velocity confidence	neighbours move together	0.0	structural prior
$\mathrm{KL}(q(z)\|\mathcal{N}(0,I))$	prior on the latent space	annealed	regulariser
KL Dirichlet	prior on state sparsity (0.25)	1.0	regulariser
TF L_1	prior on network sparsity	0.01	regulariser

6.3 The complete objective

Table 2 lists every term and, crucially, what supervises it.

Two observations follow from the table. First, only the top three terms are anchored to data; everything below is an *assumption being imposed*, not a fact being fitted. If such a prior is wrong for a given dataset — if cells genuinely move incoherently, for instance — those terms will bias the result. This is a real limitation and is best stated plainly. Second, the time reconstruction term is *self-distillation*: one part of the model generates a target for another, with a stop-gradient making the relationship one-way. Both parts are wrong at initialisation; they converge because the shared pressure to reconstruct u and s constrains them both.

6.4 Contribution 2: two complementary time supervisions

6.4.1 Loss A — indirect supervision

$$\mathcal{L}_{\text{time}} = \left\| \, t_{\text{head}} - \text{sg}\left[\, t_{\text{ODE}} \, \right] \, \right\|^2, \tag{37}$$

where t_{ODE} is the state-probability-weighted average of the four state times, averaged over genes and min–max normalised, and sg[·] denotes a stop-gradient — `.detach()` in the implementation.

What the stop-gradient does. It severs the computational graph at that point. During backpropagation the gradient flows into t_{head} but halts at t_{ODE}, so the parameters that produced the target — the decoder, and through it π, ρ, τ and the switch time t_s — receive nothing from this loss.

Why it is necessary. Objective (37) admits two descent directions, only one of which is desirable:

Route	Effect	Wanted?
move t_{head} toward t_{ODE}	the head learns the kinetics	yes
move t_{ODE} toward t_{head}	the ODE fit bends to match an MLP	**no**

The second route is destructive. t_{ODE} is derived from ρ, τ and π — precisely the quantities the reconstruction loss is fitting to the observed data. Allowing a time-agreement term to move them would degrade the ODE fit merely to make the head's task easier, and since the head is

effectively random early in training, it would drag the kinetics toward noise at the moment the fit is most fragile.

The collapse failure mode. Worse than gradual corruption, without the stop-gradient the objective admits a degenerate optimum. If both sides drift to a constant,

$$t_{\text{head}} = c \ \ \forall i, \qquad t_{\text{ODE}} = c \ \ \forall i \qquad \Longrightarrow \qquad \mathcal{L}_{\text{time}} = 0,$$

the loss is perfectly satisfied while carrying zero information: every cell receives the same time and pseudotime is destroyed. Detaching the target removes this solution, because t_{ODE} can no longer move to meet the head. The only remaining way to reduce the loss is for the head to track a quantity that is itself anchored to the data.

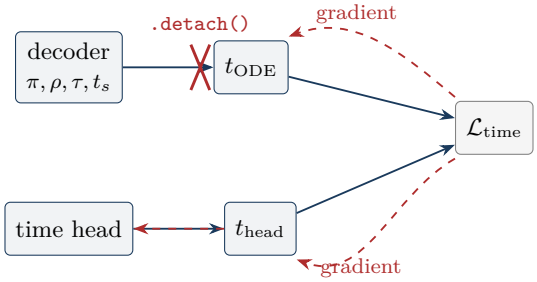

Figure 6: The stop-gradient in Loss A. Solid arrows are the forward pass; dashed red arrows are gradient flow during backpropagation. The gradient reaches t_{head} and continues into the head (and onward into z), but is severed before it can reach the decoder. Only the student is allowed to move toward the teacher.

This asymmetry — a one-directional teacher–student relationship enforced by a stop-gradient — is the same device used to prevent representational collapse in self-supervised methods such as BYOL and SimSiam, in target networks for value-based reinforcement learning, and in knowledge distillation. Here the ODE branch is the teacher, not because it is correct, but because it is the branch grounded in the reconstruction loss and therefore in the data.

6.4.2 Loss B — the ODE residual

This is the highest-impact addition. Rather than regressing the head against a derived signal, it takes the head's *own* time, scales it to $[0, t_{\max}]$, evaluates the ODE there, and compares to observation:

$$\mathcal{L}_{\text{res}} = \left\| u_{\text{ODE}}(t_i) - u_{\text{obs}} \right\|^2 + \left\| s_{\text{ODE}}(t_i) - s_{\text{obs}} \right\|^2 \tag{38}$$

The induction branch is evaluated at $\min(t_i, t_s)$ and the repression branch at $(t_i - t_s)^+$, and the two are blended by a soft gate $\sigma\big(5(t_s - t_i)\big)$ so that the branch switch is differentiable. Residuals are normalised per gene so that all genes contribute equally.

Because gradients flow into the time head — and onward into z, as noted in Section 5.5 — the head is trained to find the time that best *explains the observed expression*. This is a differentiable analogue of the expectation–maximisation time step used by scVelo, and it is the component to which the ablation attributes most of the pseudotime improvement ($0.33 \rightarrow 0.75$ Spearman).

Why there is no stop-gradient here. Loss B contains no `detach` anywhere along the path $\mathcal{L}_{\text{res}} \to t_i \to \text{head} \to z \to \text{encoder}$, and this is deliberate. The distinction from Loss A lies entirely in what the target is: here the comparison is against u_{obs} and s_{obs}, which are *observed data* — constants with no parameters behind them. There is nothing to corrupt and no degenerate solution to collapse into, so unrestricted gradient flow is exactly what is wanted. It is what allows this loss to reach the encoder and reorganise the latent space. The governing rule is therefore simple:

Detach when the target is produced by the model. Do not when the target is data.

6.4.3 The stop-gradient pattern elsewhere in the model

The same reasoning recurs in three further places, following a consistent principle: *detach the selection or the weighting, retain gradient on the quantity actually being compared.*

```
# velocity-time consistency: the pair weights are frozen
time_diff = (t_j_final - t_i_final).detach()
weights   = torch.clamp(time_diff / (time_diff.mean() + 1e-8), 0, 3)
```

Without this detach the model could reduce the loss by shrinking all latent-time gaps rather than by correcting velocity directions — satisfying the objective by manipulating the weights instead of by meeting the constraint.

```
# temporal smoothness and velocity coherence: neighbour choice is
    frozen
with torch.no_grad():
    dists = torch.cdist(z.detach(), z.detach())
    _, nn_idx = dists.topk(k, largest=False)
```

Selecting the k nearest neighbours is a discrete operation; differentiating through the choice itself is meaningless. The gradient belongs on the times and velocities being compared, not on which cells happened to be selected.

6.4.4 Geometric regularisers

Two further terms impose structure rather than fitting data. The **velocity–time consistency** loss samples random cell pairs, orders them so that $t_i < t_j$, and penalises velocity that points backwards:

$$\mathcal{L}_{\text{vtc}} = \text{ReLU}\Big(-\cos\big(v_i,\ s_j - s_i\big)\Big), \tag{39}$$

weighted by the time gap. The **temporal smoothness** loss takes the ten nearest neighbours in latent space and penalises squared time differences:

$$\mathcal{L}_{\text{ts}} = \frac{1}{k} \sum_{j \in \mathcal{N}(i)} \big(t_i - t_j\big)^2. \tag{40}$$

7 Inference

7.1 Computing velocity

For each of 25 posterior samples, the procedure is:

1. forward pass, giving $\alpha_{ig}, \beta, \gamma, \pi, \rho, \tau$;

2. evaluate the ODE at each state's time to obtain *model-predicted* u, s — not the observed counts. This is the denoising step, and it is the reason the estimate is smooth;

3. compute per-state velocities $v_{\text{ind}} = \beta u_{\text{ind}} - \gamma s_{\text{ind}}$, $v_{\text{rep}} = \beta u_{\text{rep}} - \gamma s_{\text{rep}}$, and $v_{\text{steady}} = 0$ (by definition, nothing changes at a plateau);

4. take the expectation under the state probabilities,

$$v = \pi_{\text{ind}} v_{\text{ind}} + \pi_{\text{rep}} v_{\text{rep}} + \pi_{\text{steady}} \cdot 0; \tag{41}$$

5. clip so that $v \geq -s$, ensuring predicted future abundance cannot become negative;

6. optionally smooth over the kNN graph, $v \leftarrow (1-a)v + a\,(Wv)$ with $a = 0.5$ and W row-normalised.

Averaging over posterior samples turns the sampled z into a Monte Carlo estimate of $\mathbb{E}[v]$, and the spread across samples is a usable uncertainty measure.

7.2 Three distinct notions of time

The model contains three time-like quantities which are easily confused. They live on different axes and answer different questions.

Table 3: The three time quantities.

	`learned_time` t_i	switch time t_s	velocity pseudotime t_{VPT}
Shape	$[N]$ — per *cell*	$[G]$ — per *gene*	$[N]$ — per *cell*
Range	$[0, 1]$	$[0, t_{\max}]$, with $t_{\max} = 20$	$[0, 1]$ after normalisation
Kind	variable, computed per cell	parameter, shared by all cells	post-hoc graph computation
Source	$\sigma(\text{MLP}(z))$	clamped softplus(θ_s)	diffusion pseudotime on the velocity graph
Answers	how far along the trajectory is this *cell*?	when does this *gene* stop being induced?	how far downstream of the source is this *cell*?

7.2.1 How t_i and t_s combine

The two are orthogonal, and their interaction is the mechanism by which a single cell-level clock can describe a thousand genes with different dynamics. A cell axis of length N is broadcast against a gene axis of length G:

$$\text{gate}_{ig} = \sigma\Big(5\big(t_s^{(g)} - t_{\max} \cdot t_i\big)\Big) \in [0, 1], \tag{42}$$

which asks, for every (cell, gene) pair: *has this cell's clock passed this gene's switch point?* If so the gene is repressing in that cell; if not it is still inducing.

Example. Take a cell with $t_i = 0.5$, so its clock reads 10 on the $[0, 20]$ scale.

Gene	t_s	comparison	state *in this cell*
A	5	$10 > 5$	already repressing
B	15	$10 < 15$	still inducing
C	10	$10 \approx 10$	at its peak

One cell, one clock, three different gene states — because each gene carries its own switch point. Genes turn off at different stages of the same shared trajectory, and that staggering is what constitutes a differentiation cascade.

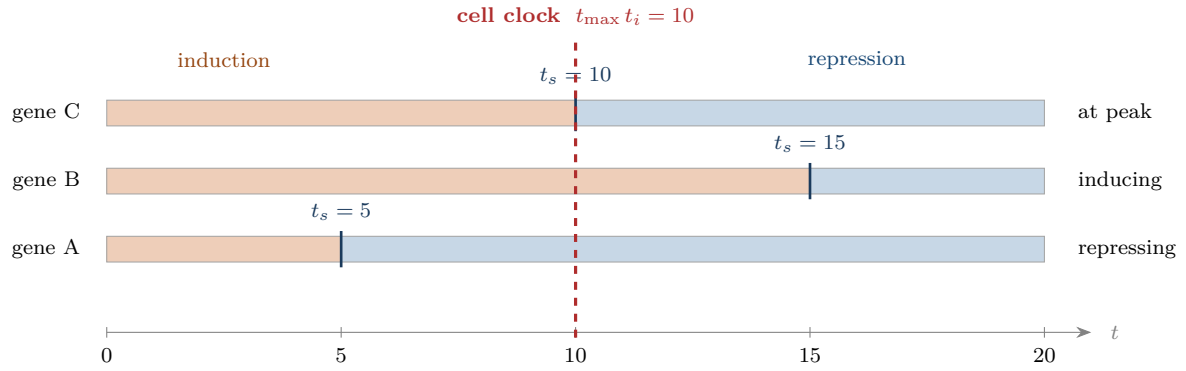

Figure 7: One cell-level clock, three gene-level switch points. The red line is this cell's time, a single number shared by every gene. Each bar is one gene's own life cycle, split at its own t_s. Where the red line falls within a bar determines that gene's state *in this cell* — which is how a single scalar per cell can describe genes with entirely different dynamics.

7.2.2 Obtaining t_i at inference

Retrieving the learned time requires only the encoder and the time head; the decoder, the ODE and the TF module are untouched, which is why it is inexpensive:

```
for tensors in scdl:                                    # minibatches of cells
    sample_times = []
    for _ in range(n_samples):                          # 25 posterior draws
        inference_outputs, _ = self.module.forward(tensors,
            compute_loss=False)
        sample_times.append(inference_outputs["learned_time"].cpu().
            numpy())
    all_times.append(np.mean(sample_times, axis=0))
```

Because z is *sampled* from $q(z \mid s, u)$ rather than taken at its mean, each pass yields a slightly different t. Averaging is Monte Carlo estimation of the posterior mean:

$$\hat{t}_i \;=\; \mathbb{E}_{z\sim q(z|s_i,u_i)}\big[\,\mathrm{head}(z)\,\big] \;\approx\; \frac{1}{25}\sum_{k=1}^{25}\mathrm{head}\big(z^{(k)}\big). \tag{43}$$

7.3 Contribution 3: velocity–pseudotime blending

7.3.1 Computing t_{VPT}

Velocity pseudotime is a cell ordering derived from the velocity field itself. It is computed after training, as post-processing, in four steps.

Step 1: the velocity graph. For every cell i and each neighbour j, compare where cell i is pointing against where cell j actually lies:

$$\pi_{ij} = \cos\big(v_i,\; s_j - s_i\big) = \frac{v_i \cdot (s_j - s_i)}{\|v_i\|\,\|s_j - s_i\|}. \tag{44}$$

A high value means i's velocity points directly at j, making j a plausible next state. The resulting matrix is *asymmetric*, and that asymmetry is the entire source of directionality.

Step 2: transition probabilities. Exponentiate and row-normalise over neighbours,

$$P_{ij} = \frac{\exp(\sigma \pi_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(\sigma \pi_{ik})}, \tag{45}$$

turning the velocity field into a directed Markov chain over cells.

Step 3: diffusion pseudotime. A root cell is inferred from the spectral structure of P — roughly, the cell from which flow originates — and diffusion pseudotime is computed as the diffusion distance from that root. The essential point is that the ordering derives from *velocity directions*: ordinary diffusion pseudotime on a kNN graph yields an undirected ordering with no way to identify which end is the beginning. Velocity supplies the arrow of time.

Step 4: normalisation and blending. The result is scaled to $[0, 1]$, the learned time is sign-aligned against it, and the two are averaged:

```
corr, _ = spearmanr(learned_t, vpt_norm)
if corr < 0:
    learned_t = 1.0 - learned_t          # sign alignment
latent_time = blend * vpt_norm + (1 - blend) * learned_t   # blend =
    0.5
```

The sign alignment is necessary because a pseudotime without a root is defined only up to reversal; t_{VPT} has an inferred root, so its direction is treated as canonical.

7.3.2 Why blending helps

	Global direction	**Local smoothness**
t_{VPT}	yes — has a root, follows the flow	no — jagged on a sparse noisy graph
`learned_time`	no — can drift without an anchor	yes — a smooth function of z

Each component compensates for the other's failure mode. The two are produced by entirely independent mechanisms — a per-cell regression and a global diffusion process — so their errors are largely uncorrelated and averaging cancels noise rather than reinforcing it.

Note also that t_{VPT} is computed *from* TARVI's velocity, so the quality of Contribution 3 depends directly on Contributions 1 and 2. The three do not merely add; they compound.

8 Evaluation

Training uses no ground truth. Evaluation deliberately does, and holds it out entirely from the fitting procedure:

- **Cell-type annotations** support cross-boundary direction accuracy (CBDir), which checks that velocity points from progenitor to descendant across known lineage boundaries.
- **Known developmental ordering** allows pseudotime to be scored by rank correlation.
- **Measured time** exists for the scEU-seq organoid dataset via metabolic labelling, and is consumed directly by the ground-truth temporal metrics.

Additional metric families cover velocity confidence and coherence length, gene-level velocity R^2, state-assignment entropy and purity, and transition probability scores. Benchmarking is against velocyto, scVelo in both deterministic and dynamical modes, and VeloVI, across five datasets, with a four-mode component ablation.

The methodological point is that ground truth is available but is *deliberately excluded* from training. Had the labels been used to fit the model, the benchmark numbers would carry no information.

Summary of results

Table 4: Cross-dataset means over five datasets. Higher is better throughout.

Method	CBDir	VelConf	PT-Spear	PT-DistCorr	PT-Cons	Gene-R^2
TARVI	**0.933**	**0.927**	**0.747**	**0.771**	**0.986**	**0.477**
VeloVI	0.931	0.920	0.331	0.341	0.904	0.443

TARVI matches the strongest baseline on cross-boundary direction accuracy while substantially improving pseudotime calibration, a 126% relative gain in Spearman correlation over VeloVI.

A Notation and tensor shapes

Symbol	**Shape**	**Meaning**
N, G, T	—	number of cells, genes, transcription factors
u, s	$[N, G]$	unspliced and spliced abundance (moments, scaled)
α	$[N, G]$	transcription rate; cell-specific under TF regulation
β, γ	$[G]$	splicing and degradation rate, per gene
v	$[N, G]$	RNA velocity, $\beta u - \gamma s$
z	$[N, 10]$	latent cell state
μ_z, σ_z^2	$[N, 10]$	variational posterior parameters
π	$[N, G, 4]$	kinetic state weights (Dirichlet)
ρ	$[N, G]$	fractional progress through induction
τ	$[N, G]$	fractional progress through repression
t_s	$[G]$	per-gene switch time
$t_{\max}$	scalar	time horizon, fixed at 20
t_i	$[N]$	learned per-cell latent time, in $[0, 1]$
t_{VPT}	$[N]$	velocity pseudotime
M	$[G, T]$	binary TF–target mask from ENCODE and ChEA
W	$[G, T]$	learnable TF–target weights
b	$[G]$	basal transcription rate

B Where each component lives in the code

Component	Location
Preprocessing, kNN velocity smoothing	`tarvi/_utils.py`
TF mask construction, cell-specific α	`tarvi/_tf_module.py`
Encoder, decoder, ODE solutions, all losses	`tarvi/_module.py`
Model class, velocity and time getters, VPT	`tarvi/_model.py`
Graph-attention velocity refiner	`tarvi/_gnn.py`
Benchmark metrics	`evaluation/metrics.py`
Training, benchmarking, ablation scripts	`scripts/`